A Beginner's Guide to Relational Databases in Lean

Jaisuraj Kaleeswaran

June 13, 2026

1 Motivation and Background

Relational databases are usually introduced as a practical technology: tables store tuples, queries select or filter tuples, and joins combine tuples. At the same time, the relational model is a mathematical theory. Tables can be seen as relations, query predicates as logical predicates, and optimizations to queries can be proved using equivalences between expressions. Therefore, databases are a natural way of connecting application science to formal reasoning.

The purpose of this project is to make this connection visible to students who already have some programming and database knowledge but only limited experience with Lean or with interactive proving. While many beginner tutorials on proof assistants are geared towards proofs in arithmetic, logic puzzles or abstract mathematics, these often present proof assistants as tools disconnected from systems concerns. Instead, I develop a simplified database scenario, consisting of products, product comments, customers, and orders.

This project aims to provide an answer to a simple question: How do we model a miniature relational database in Lean and use it to prove small facts about database operations? The response is deliberately minimalist: I do not aim to write a full database engine or an entirely complete formalization of SQL; rather, I provide an small demonstration tool that shows how common database operations can be formalized as definitions in ordinary Lean and then studied using Lean proofs. Therefore, the result of this work is not intended to be a library for production use but rather a verifiable Lean file along with a descriptive report.

So why is this important? It gives those who know Lean but not SQL and relational database an idea of how to do it formally. Join operation no longer has to be just some low-level implementation detail but rather a formal operation. Once you have a formal definition of an operation, you can prove a few trivialities about it. Such as joining any table with an empty table is always an empty table. Or filtering a table with an always-true predicate is always identical to no operation on the table. While these examples might be too small, they demonstrate a more general idea of verifiable query optimization and formal data reasoning.

2 Design Overview and Theoretical Foundations

The design is based on a simplified version of the relational model. A relation is represented as a list of rows, and a row is represented as either a Lean record or a tuple. The code uses

records for the main examples because field names make the tutorial easier to read. For example, a product row has an ISBN, a title, and a price. A product comment row has an ISBN and a comment. Matching ISBNs then provides a natural example of a join key.

The Lean companion file defines four main row types:

- `Product`, with fields `isbn`, `title`, and `price`.
- `ProductComment`, with fields `isbn` and `comment`.
- `Customer`, with fields `customerId` and `name`.
- `Order`, with fields `orderId`, `customerId`, `isbn`, and `quantity`.

Relational algebra operators are defined as functions on lists. Selection is list filtering. Projection is list mapping. Cartesian product creates a list of all pairs from two lists. The definition of inner join is to take a Cartesian product of the two input tables, then filter the pairs in the product using the condition, and then map the filtered pair to an output row.

This is not the only possible model for relational databases; a purely mathematical model could be to take a relation to be a set of tuples, have types and enforce schemas at the type level and proves that primary keys have the uniqueness property. Lists are used because it's an object that is very familiar to programming novices, that the order in the result lists is clear when running examples and that lists can be evaluated efficiently using `eval`. This helps to achieve the pedagogical purpose, that of clearly exhibiting both the computational part and the formal proof of the relation algebra operation. The disadvantage of using list semantics is that duplicate tuples are permitted unless further specified otherwise.

Here's the connection between database operations and functions:

- Selection corresponds to applying a predicate to each row and keeping the rows that satisfy it.
- Projection corresponds to mapping each row to a smaller or differently shaped row.
- Cartesian product corresponds to pairing every row from one table with every row from another table.
- Join corresponds to a Cartesian product followed by selection and projection.

This representation also helps explain Lean's role. Lean can be used to check definitions as programs or to use the definitions in a theorem statement. The same definition of innerJoin is useful in a demo query and in a theorem relating to joins with empty tables. By having Lean as both a programming language and a theorem prover, the artifact treats relational algebra in two related ways. One is operational: a query is a program to translate from one list to another. The other is logical: the same query is an expression whose effects can be stated in a theorem. The fundamental theoretical concept in the project is this dual treatment; rather than formalization and implementation being separate activities, the definitions checked by Lean can also be considered executable.

3 Implementation Details

The implementation is located in `RelationalDatabasesInLean.lean`. It doesn't have any dependencies and can be checked with the standard Lean executable. I deliberately made the decision to keep the whole implementation in a single file, for the reason that it is more approachable for an introductory tutorial to examine, run, and modify a simple project with no external dependencies compared to a large project with numerous dependencies. The file is laid out to be easy to read in a guided sequence: it starts with definitions of data structures, followed by explicit values of table entries, definitions of relational operators, demonstration of the execution of example queries, and finally proofs of properties about the implementation. I want students to have the same intuitive learning experience, reading about rows, then tables, then operations over tables, then theorems about the operation. The tutorial doesn't force the reader to understand Lean syntax prior to understanding the database operations.

The first part of the file defines the row structures and toy data. Here is a representative excerpt:

```
structure Product where
  isbn : Nat
  title : String
  price : Float
deriving Repr

structure ProductComment where
  isbn : Nat
  comment : String
deriving Repr
```

The deriving `Repr` clause makes it possible for Lean to print out row structures with the `eval` command. This can be used as an pedagogical tool: the student can immediately see the output of executing a query.

The Cartesian product operation is defined recursively on lists:

```
namespace List

def outer {alpha beta : Type _}
    (xs : List alpha) (ys : List beta) : List (Prod alpha beta) :=
  match xs with
  | [] => []
  | x :: xs' => (ys.map fun y => (x, y)) ++ xs'.outer ys

end List
```

This definition mirrors the database intuition: for each row in the first table, pair it with every row in the second table. The remaining operations are short wrappers around list functions:

```
def select {alpha : Type _} (p : alpha -> Bool)
    (rows : List alpha) : List alpha :=
  rows.filter p

def project {alpha beta : Type _} (f : alpha -> beta)
    (rows : List alpha) : List beta :=
  rows.map f
```

The inner join definition combines these pieces:

```
def innerJoin {alpha beta gamma : Type _}
    (matchRows : alpha -> beta -> Bool)
    (combine : alpha -> beta -> gamma)
    (xs : List alpha)
    (ys : List beta) : List gamma :=
  xs.outer ys
    |>.filter (fun pair => matchRows pair.1 pair.2)
    |>.map (fun pair => combine pair.1 pair.2)
```

The implementation uses a Boolean matching predicate rather than a proposition-valued predicate. This keeps the operation executable and easy to demonstrate with `#eval`. For example, joining products and comments by ISBN is written as:

```
def productCommentPairs : List (Prod String String) :=
  innerJoin
    (fun p c => p.isbn == c.isbn)
    (fun p c => (p.title, c.comment))
    products
    comments
```

This design shows an interesting aspect of Lean: the same functionality can be viewed in a number of ways – as an executable function, as a type-correct piece of code, as something within a proof – this is the place where InfoView is useful to make the expected type and goal to proof apparent as the user writes code and proof.

One implementation choice: I used Boolean valued predicates instead of proposition valued predicates. A proposition valued predicate might be conceptually clearer since it is "closer to a proof, but still within a term" (as of my writing, although I acknowledge this phrasing might be confusing. The reason for using Boolean valued predicates is that `filter` expects it and they make more sense to an introductory student. However, as a teaching opportunity, we could explore proposition valued predicates more deeply later by discussing the difference between executable and proof-based computation using an example like the join predicate `fun p c => p.isbn == c.isbn` which looks like code that a student accustomed to programming would be able to read and comprehend easily..

I define the Cartesian product step-by-step manually (instead of defining join using Cartesian product), to make its implementation clearer to the student. This is because join has no external definitions and relies on cartesian product, filter, and projection in that order, thus

a step-by-step definition clearly mimics the structure of a relation algebra join and gives a clean path from the relational algebra and theory to the lean code.

The file also includes a small syntax experiment that makes selection and projection look closer to a database query. The macro introduces a lightweight form:

```
def productsUnder50' : List String :=
  from p in products,
    select p.price < 50.0,
    project p.title
```

This expands to the same pipeline used elsewhere:

```
products |> select (fun p => p.price < 50.0) |> project (fun p => p.title)
```

This macro is intentionally narrow. It is not meant to parse SQL, support arbitrary clauses, or hide the underlying Lean code. Its purpose is to show that Lean's syntax extension system can be used to create beginner-friendly notation around a checked core. That makes the implementation more useful as a tutorial: students can first read a query-like expression and then inspect the ordinary functional program it expands to.

The proof examples use simple tactics such as `rfl`, `induction`, `simp`, and `native_decide`. For instance, I've included an example for an empty join, and another two for a condition in selection being always true or always false. My proof for the example query combines evaluation and proof: it uses computation to determine whether the expected output is indeed equal to the result from the query, thus presenting multiple pathways for a student to enter the field of Lean proof development with a minimal amount of mathematical prerequisites.

4 Tutorial Design and Deliverables

The tutorial aspect of this project has been created to address a particular audience. My tutorial is geared towards people familiar with Lean and with the Lean syntax, who however, do not know anything about records, lists, filtering or any databases at all. It does not assume anything about knowledge of proof assistants and learning is obtained by the use of output from Lean only.

The first section of the tutorial asks the user to inspect the structures of rows. No explanation about what `Product` actually is provided; instead, the student is asked to realize it is a natural record with attributes and then he can build upon this information when relational algebra will be explained. After that, how tables are represented by a list of records will be presented. It is not a realistic approach, but lists have very simple operations, and they are visually very intuitive and this is suitable for the target audience.

After defining the data model, the tutorial discusses selection and projection by example. The concepts of projection are presented simply as "choose the output columns" despite the fact that the implementation in Lean is technically a function that maps rows of one structure to rows of another structure. Similarly, selection is explained simply as "select rows with this property." The aim here is to make functions in Lean feel like operations in a database context.

The join section is the center of the tutorial. The general concept that a join merges matching rows from two tables is introduced. From here, the construction of the Cartesian product is discussed, from which elements are selected based on whether the key column is the same in each table and from which results are obtained by mapping from two rows to one output row. It is intended that this gradual approach, beginning with Cartesian product before adding filtering and then mapping, will convey the fact that the join is built from a composition of functions rather than being an inherent operation like one may feel with a built-in database feature.

The proof portion of the tutorial is brief, yet intended to be meaningful. It is designed to introduce the user to the style of formal reasoning rather than to complex database theorems. A statement like "selection with an always true condition will preserve the input table" is a readily understandable theorem that introduces the user to theorem statements, proof goals, and proof scripts. It is at this point that the Lean InfoView is introduced, as the user is prompted to observe the changes in the goal as it is reduced by induction and by simplification.

The deliverables for this project are the Lean companion file and the present report. The Lean companion file has been ordered such that it may be used as a tutorial on its own, with sections ranging from definitions through examples and finishing with proof sections, and such that each main definition is small enough to discuss verbally when being presented. The report can be read alongside, providing an explanation for why the artifact has been constructed in this way and how it relates to the theory of relational algebra.

Also notable are the intentionally omitted portions. Real-world databases contain schemas, constraints, indexes, query planners, transactions and more besides; it is the intention of this project not to obscure the realities of working with databases, but to postpone discussion until a base understanding of definitions, execution and formal specification has been achieved. The primary goal of this artifact is to demonstrate the possibility of giving database operations precise definitions and formal specification that can be tested.

5 Evaluation and Results

The project was evaluated in two ways: by running the example queries and by checking the proof examples. The Lean file checks successfully with Lean 4.30.0. Running `lean RelationalDatabasesInLean.lean` evaluates the demonstration queries and checks the theorems. This gives a useful baseline: the code is not only illustrative text in the report but also a mechanically checked artifact.

The presentation demo can follow the same order as the file. First, I can show the row structures and explain how they correspond to database tables. Second, I can run the projection and selection examples to show that Lean is executing the definitions. Third, I can run the product-comment join and compare the result to the expected database intuition. Finally, I can show one or two theorem statements and use the InfoView to explain the current proof goal. This keeps the demo focused on the project's main message: the same artifact supports programming, querying, and proving.

The first examples show projection and selection. Projecting product titles produces:

```
["Databases in Lean", "History of AI", "Query Plans and Proofs"]
```

Selecting products whose price is under 50 and projecting their titles produces:

```
["Databases in Lean", "Query Plans and Proofs"]
```

The query-like macro produces the same result for the same selection-and-projection example:

```
["Databases in Lean", "Query Plans and Proofs"]
```

This result is useful in the demo because it shows two views of the same query. The pipeline version emphasizes that relational operations are ordinary functions. The macro version emphasizes that a more database-like surface syntax can be built on top of those functions without changing the core semantics.

The main join example combines products with comments by ISBN:

```
[("Databases in Lean", "Awesome!"),
 ("Databases in Lean", "Do not buy, overhyped."),
 ("History of AI", "Only covers September 2018, avoid.")]
```

A second join example combines customers and orders by customer ID:

```
[("Ada", 123894523, 2), ("Ada", 991122333, 1),
 ("Grace", 534432123, 1)]
```

The file also contains several checked proof examples. Some are general facts:

- The Cartesian product of an empty left table with any table is empty.
- The Cartesian product of any table with an empty right table is empty.
- Selecting with an always-true predicate preserves a table.
- Selecting with an always-false predicate returns an empty table.
- Inner joining with an empty left or right table returns an empty result.
- Projecting an empty table returns an empty table.

One proof checks a concrete query result:

```
theorem concreteCommentJoin :
    productCommentPairs =
      [ ("Databases in Lean", "Awesome!")
      , ("Databases in Lean", "Do not buy, overhyped.")
      , ("History of AI", "Only covers September 2018, avoid.")
      ] := by
  native_decide
```

As seen by this theorem, we can show with Lean that the query over the toy database does return the correct value. Though this example is small, it captures the important notion that "expected query behavior" can be specified as a theorem and checked mechanically.

Limitations of the current artifact: The model uses lists and therefore duplicate rows can exist. Primary keys are implemented as fields such as ISBN or customerId, but we have not yet shown how to prove the uniqueness of such keys. Joins are currently only inner joins, and the tutorial has not attempted to formalize SQL syntax, null values, aggregation, indexing, query planning, transactions, etc. All of these omissions are intentional, as our goal has been to create an introduction rather than a comprehensive database formalization.

Educational successes: My project will be considered a success of this artifact if a user can answer these three questions after working through the tutorial: (1) How to model tables in Lean? (2) How to specify operations in Lean as typed functions? (3) How to state simple facts about operations as theorems? The current code is small enough to present the answers to all three questions in one class period or one tutorial.

Practical successes: One type of example that the current artifact handles is a simple inner join keyed over two small tables. A key-based inner join such as the product-comment join (where we see one-to-many relationships since we get two rows for each product comment) or customer-order join (we join the current customer data with transactional order data) can be implemented. A SQL example where the artifact fails is one involving null values, group-by, aggregate functions, or SQL's notion of duplicates in relations. For instance, `SELECT DISTINCT` or `GROUP BY` require additional definitions and theorems, thus clearly defining the extent of what the current project enables (verified relational reasoning).

6 Discussion and Reflection

The most important lesson from the project is that Lean can make ordinary database ideas feel more precise without requiring a large system. Selection, projection, Cartesian product, and join are familiar operations from relational algebra, but defining them in Lean forces every input, output, and predicate to have a type. That explicitness is useful for students because it turns vague descriptions like "join two tables" into a concrete function with a clear interface.

Another lesson is that executable examples and proofs support each other. The `#eval` examples make the definitions feel concrete. The theorem statements show that the same definitions can be used for verification. For a beginner, this is a productive bridge: first run the query, then ask what property the query should satisfy.

I also learned that choosing the right abstraction level is important. A more realistic model of relational databases would include schemas, constraints, set semantics, and query equivalence laws. However, that would add significant complexity for a first tutorial. Lists of records are less mathematically complete, but they are approachable and easy to connect to code. For the intended audience, that tradeoff is worthwhile.

One challenge was deciding how much database terminology to encode in Lean types. It would be possible to make schemas, attributes, keys, and constraints explicit in the type system. That direction is attractive because it could prevent malformed queries earlier. However, it would also make the first example much harder to read. In this project, I

kept the types close to ordinary records and moved the database interpretation into the explanatory text. That made the code less general, but it made the tutorial more accessible.

Another challenge was balancing automation and explanation in the proofs. Lean can discharge some small goals with tactics such as `simp` or `native_decide`, but a beginner may not learn much from a proof if it looks like a black box. I therefore included proofs where the structure is visible, especially induction over lists, alongside proofs that use computation. This combination reflects how Lean is used in practice: automation is valuable, but the user still needs to understand the definitions and the proof goals.

If I continued the project, I would extend it in several directions. First, I would add explicit primary-key uniqueness predicates and prove that the toy data satisfies them. Second, I would add more query equivalence theorems, such as laws about pushing selections before joins. Third, I would create a web version of the tutorial, ideally using a Lean-aware documentation system such as Verso so that the code examples could be checked as part of the site generation process. Finally, I would add guided screenshots or explanations of the InfoView so students can see how expected types and proof goals help them write Lean code.

Overall, the project shows that a small Lean file can serve as a useful educational bridge between relational databases and formal verification. It does not replace a database systems course or a theorem-proving course, but it gives students a concrete starting point for seeing how the two areas can inform each other.

You can access my repo here: `https://github.com/jaisurajk/RelationalDatabasesinPL`

7 References

- E. F. Codd. "A Relational Model of Data for Large Shared Data Banks." *Communications of the ACM*, 1970.
- The Lean 4 documentation. `https://docs.lean-lang.org/`
- Theorem Proving in Lean 4. `https://lean-lang.org/theorem_proving_in_lean4/`
- The Lean Language Reference. `https://lean-lang.org/doc/reference/latest/`
- Verso documentation authoring tool. `https://github.com/leanprover/verso`